IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 29 November 2023, accepted 20 January 2024, date of publication 24 January 2024, date of current version 12 February 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3358201

RESEARCH ARTICLE

Software Defect Prediction Using an Intelligent Ensemble-Based Model

MISBAH ALI[1], TEHSEEN MAZHAR[1], YASIR ARIF[2], SHAHA AL-OTAIBI[3], (Member, IEEE), YAZEED YASIN GHADI[4], TARIQ SHAHZAD[5], MUHAMMAD AMIR KHAN[6], AND HABIB HAMAM[5,7,8,9], (Senior Member, IEEE)

[1]Department of Computer Science, Virtual University of Pakistan, Lahore 55150, Pakistan
[2]Department of Computer Science, Global Institute, Lahore 54000, Pakistan
[3]Department of Information Systems, College of Computer and Information Sciences, Princess Nourah bint Abdulrahman University, P. O. Box 84428, Riyadh 11671, Saudi Arabia
[4]Department of Computer Science and Software Engineering, Al Ain University, Abu Dhabi, United Arab Emirates
[5]School of Electrical Engineering, Department of Electrical and Electronic Engineering Science, University of Johannesburg, Johannesburg 2006, South Africa
[6]School of Computing Sciences, College of Computing, Informatics and Mathematics, Universiti Teknologi MARA, Shah Alam, Selangor 40450, Malaysia
[7]Faculty of Engineering, University of Moncton, Moncton, NB E1A3 E9, Canada
[8]International Institute of Technology and Management (IITG), Commune d'Akanda, Libreville 1989, Gabon
[9]Bridges for Academic Excellence, Tunis, Centre Ville 1002, Tunisia

Corresponding authors: Muhammad Amir Khan (amirkhan@uitm.edu.my) and Tehseen Mazhar (tehseenmazhar719@gmail.com)

This work was supported by Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia, through the Princess Nourah bint Abdulrahman University Researchers Supporting Project under Grant PNURSP2024R136.

ABSTRACT Software defect prediction plays a crucial role in enhancing software quality while achieving cost savings in testing. Its primary objective is to identify and send only defective modules to the testing stage. This research introduces an intelligent ensemble-based software defect prediction model that combines diverse classifiers. The proposed model employs a two-stage prediction process to detect defective modules. In the first stage, four supervised machine learning algorithms are employed: Random Forest, Support Vector Machine, Naïve Bayes, and Artificial Neural Network. These algorithms are optimized through iterative parameter optimization to achieve the highest accuracy possible. In the second stage, the predictive accuracy of the individual classifiers is integrated into a voting ensemble to make the final predictions. This ensemble approach further improves the accuracy and reliability of the defect predictions. Seven historical defect datasets from the NASA MDP repository, namely CM1, JM1, MC2, MW1, PC1, PC3, and PC4, were utilized to implement and evaluate the proposed defect prediction system. The results demonstrate that each dataset's proposed intelligent system achieved remarkable accuracy, outperforming twenty state-of-the-art defect prediction techniques, including base classifiers and ensemble methods.

INDEX TERMS Machine learning, software defect prediction, ensemble classification, heterogeneous classifiers, random forest, support vector machine, naïve Bayes.

I. INTRODUCTION

Rapid globalization has transformed our world into a closely connected village where the software industry is crucial in driving progress. In this age of a digitally interconnected world, software applications have emerged as the lifeblood of our global society, forming the backbone of daily activities, businesses, and critical infrastructure [1], [2]. This influence has only intensified, particularly after the COVID-19 pandemic, which has accelerated our reliance on online systems for communication, commerce, and remote work [3], [4], [5].

In a Software Development Life Cycle (SDLC), the work flow from the development team to the Quality Assurance (QA) team typically involves several stages. Initially, the development team implements the software code handed over to the QA team for testing. The QA team then rigorously evaluates the software, identifying and reporting defects or issues. The iterative feedback loop between development and QA continues until a high-quality, defect-free software

The associate editor coordinating the review of this manuscript and approving it for publication was Juan A. Lara.

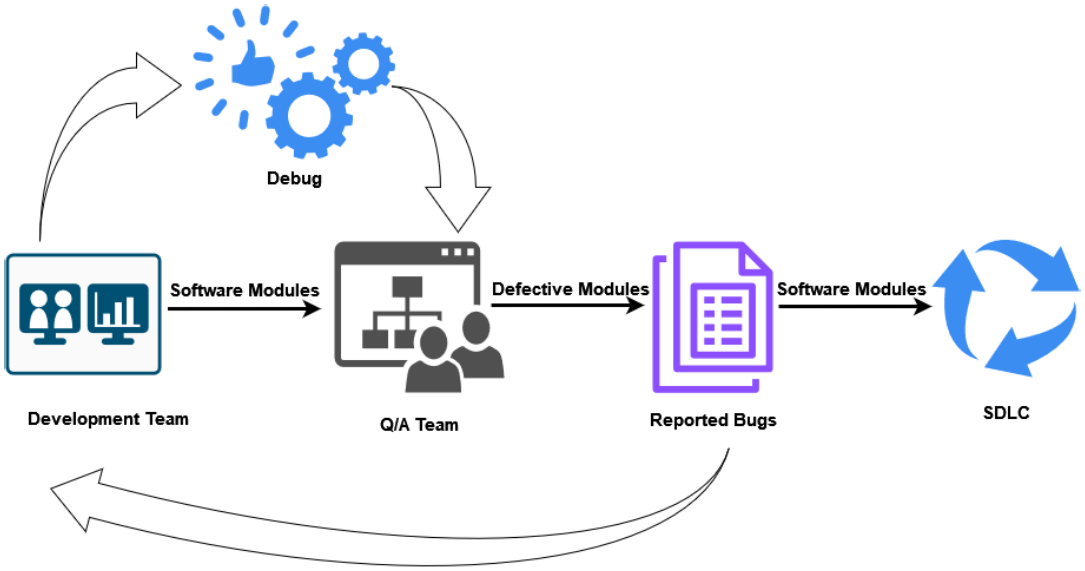

FIGURE 1. Development-to-QA workflow in SDLC.

product is achieved [6], [7]. The workflow from the Development-to-QA team in an SDLC is shown in Figure 1. However, defect-free software is not without its challenges. Three critical factors that prominently influence software quality assurance are time, financial resources, and the availability of skilled manpower. The industry's ever-growing demand necessitates formulating effective testing strategies to optimize these valuable resources while maintaining the highest software quality standards [8].

This is where software defect prediction (SDP) steps into the spotlight. SDP is the art of leveraging historical data and machine learning (ML) techniques to forecast and identify potential defects in software systems before the testing phase [9]. It investigates the complex set of software metrics such as code complexity, size, and historical defect data to build models capable of gauging the likelihood of defects [10].

The integration of software defect prediction disrupts the traditional development-to-QA workflow. The feedback loop is altered by proactively identifying potential defects before the testing phase. The predictive insights allow developers to address and rectify potential issues before the software reaches the QA team. This process streamlines the process and reduces the traditional back-and-forth between development and QA. This shift promotes a more efficient and cost-effective software development life cycle [11]. A visual representation highlighting the reduced feedback loop when the SDP model is in place is shown in Figure 2.

In the context of software defect prediction, classification techniques take centre stage. They involve categorising data into classes or labels, making them particularly relevant in identifying potential software defects. Various classification techniques include decision trees, logistic regression, support vector machines, etc [12], [13]. These techniques aid in assessing and addressing software quality concerns proactively. Historically, several studies have utilized the power of classification techniques to enhance the accuracy of defect prediction models. However, prior work in this field has limitations, including challenges related to classification techniques, overfitting, and underfitting [14].

Parallel to classification, Ensemble Modeling (EM) has also emerged as a promising technique combining the predictions from multiple ML models to improve overall performance [15]. Ensemble methods, such as bagging, boosting, stacking, and random forest, contribute to the field by mitigating inherent challenges [15]. By aggregating the predictions of multiple base classifiers, ensemble techniques reduce the risks of overfitting, underfitting, and biases that can affect individual classifiers. They have proven to be valuable in enhancing the accuracy and robustness of defect prediction models by reducing the inherent biases of individual classifiers [16], [17]. Nevertheless, researchers have encountered a standard stumbling block: the inherent susceptibility of ensemble techniques to biases that can influence their efficacy [18], [19]. Figure 3 presents an overview of the software defect prediction using ML techniques.

To meet the ever-pressing need for cost-effective, high-quality software, the modern software development paradigm must be equipped with an SDP system that is both effective and efficient. This research unveils an intelligent ensemble-based software defect prediction model that combines the strengths of diverse classifiers and ensemble techniques while optimizing resource utilization cost-effectively.

The proposed model, known as the Voting Ensemble-Based Software Defect Prediction model (VESDP), integrates four heterogeneous supervised classifiers: Random Forest (RF), Support Vector Machine (SVM), Naïve Bayes (NB), and Artificial Neural Network (ANN). Through iterative tuning, each classifier is optimized for maximum accuracy, thereby elevating the model's predictive performance. The predictive accuracy of these diverse classifiers is further integrated into

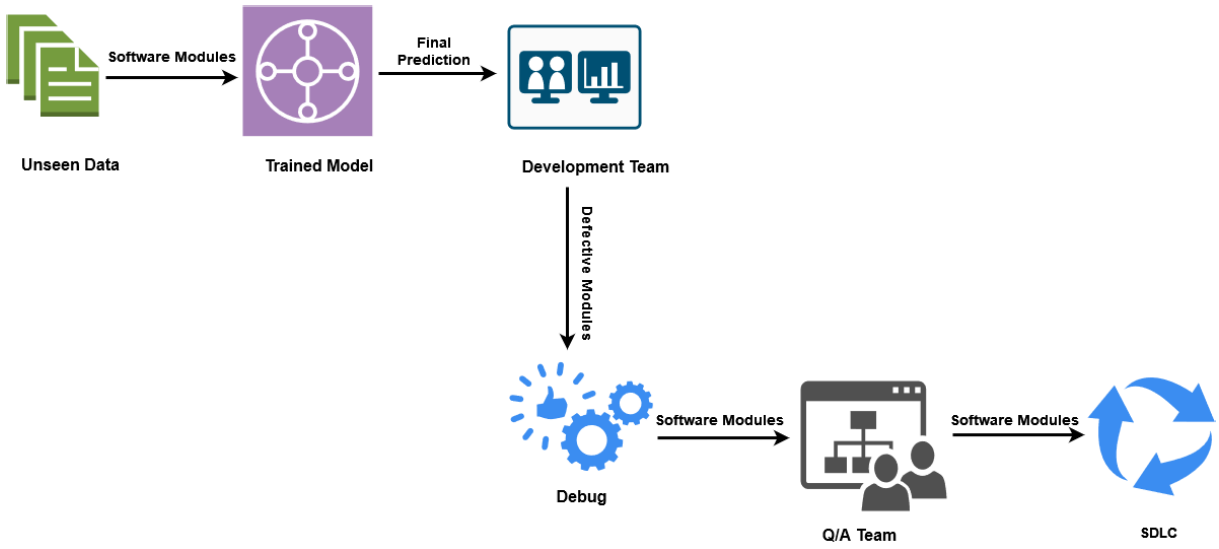

FIGURE 2. Development-to-QA workflow using SDP.

a voting ensemble, offering a remedy for the biases often encountered when individual classifiers are relied upon in isolation.

The VESDP model's performance is rigorously evaluated using seven datasets extracted from the NASA MDP repository. In this evaluation, a comprehensive suite of eight performance measures, including Predicted Positive Value (PPV), Predicted Negative Value (PNV), True Positive Rate (TPR), True Negative Rate, accuracy, Misclassification Rate (MR), False Positive Ratio (FPR), and False Negative Ratio (FNR), are employed to measure the model's efficacy and robustness. The results conclusively demonstrate the higher predictive power of the proposed VESDP model, achieving remarkable accuracy across all datasets and surpassing state-of-the-art techniques in the field.

A. OBJECTIVE OF THE STUDY

This study is aimed at achieving the following research objectives

1. Create a software defect prediction model that combines diverse classifiers using a voting ensemble technique
2. Assess the model's accuracy using historical defect datasets and eight performance measures
3. Compare the model's performance with existing techniques to demonstrate its effectiveness

B. MOTIVATION OF THE STUDY

The study explores ensemble models in SDP, aiming to enhance predictive performance, stability, and robustness [20]. It investigates the impact of heterogeneous supervised machine learning classifiers on software defect prediction models. Additionally, the research conducts a comparative analysis with twenty state-of-the-art techniques to establish the effectiveness of the proposed framework, validate its novelty, and provide valuable insights for the research community and industry. This approach contributes to progress in the field by serving as a benchmark for evaluating and fostering the development of advanced solutions [21], [22].

C. CONTRIBUTION OF THE STUDY

The contributions of this research can be summarized as follows

- Novel Ensemble-Based Model: We introduce a pioneering ensemble-based software defect prediction model (VESDP) that proficiently combines classification and ensemble techniques, pushing the boundaries of traditional approaches.
- Thorough Evaluation with Diverse Datasets: We rigorously evaluate the VESDP model by subjecting it to testing with seven carefully selected datasets from the NASA MDP repository, offering a robust foundation for assessment.
- Remarkable Accuracy: Our VESDP model attains exceptional accuracy rates across all datasets, clearly surpassing established state-of-the-art techniques in the field, underscoring its groundbreaking potential.
- Quantified Impact of Ensemble Technique: We quantify the tangible impact of integrating predictive accuracy from diverse classifiers through the voting ensemble technique, revealing its effect through a comprehensive analysis of multiple performance measures.
- Benchmarking Against Modern Methods: We conduct an extensive comparative analysis, comparing the proposed VESDP model against twenty published techniques, highlighting its excellence in the realm of software defect prediction

D. ORGANIZATION OF THE STUDY

The rest of the paper is organized as follows: Section II thoroughly reviews existing literature, while Section III outlines the proposed VESDP framework, detailing its phases

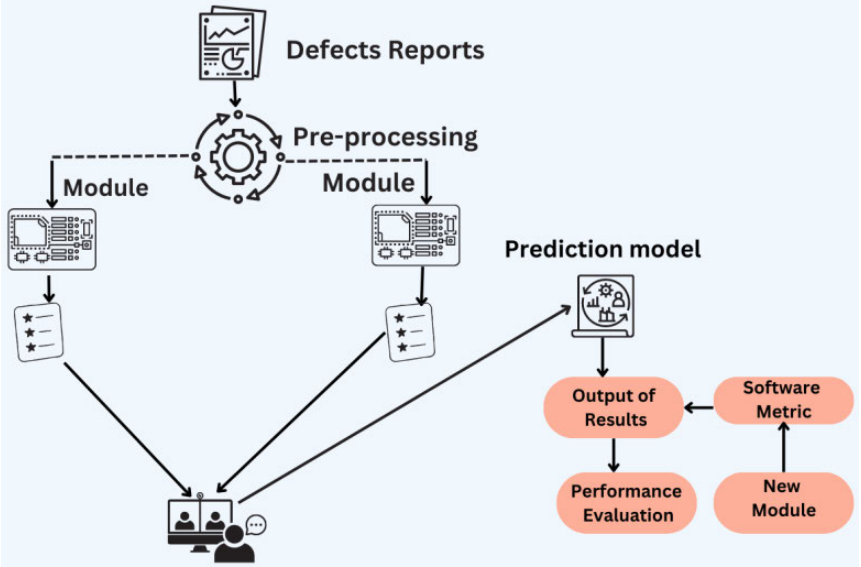

FIGURE 3. Software defect prediction process.

TABLE 1. List of abbreviations.

Abbreviations	Description	Abbreviations	Description	Abbreviations	Description
ML	Machine learning	ANN	Artificial Neural Network	SDP	Software defect prediction
SVM	Support Vector Machine	FPR	False Positive Ratio	FNR	False Negative Ratio
NB	Naïve Bayes	DT	Decision Tree	EM	Ensemble Modeling
PPV	Predicted Positive Value	PNV	Predicted Negative Value	TPR	True Positive Rate
VESDP	Voting ensemble-based software defect	EXTRF	Extended Random Forest	GA	Genetic Algorithms
SSL	Semi-Supervised Learning	KNN	K-Nearest Neighbor	MLP	Multi-Layer Perceptron
SMOTE	Minority Oversampling Technique	SDLC	Software Development Life Cycle	TNR	True Negative Rate
MR	Misclassification Rate	AUC	Area Under the Curve	ELM	ELM Extreme learning Machines
LR	Linear Regression	PLS	Partial Least Square	HGB	HIST Gradient Boosting

and activities. Section IV presents results and a comparative analysis of the VESDP framework against state-of-the-art techniques. Section V addresses potential research validity concerns, and Section VI delivers a summary, findings analysis, and suggestions for future research.

II. LITERATURE REVIEW

A. CLASSIFICATION

Authors in [3] developed an intelligent cloud-based SDP system incorporating data fusion and decision-level machine learning fusion techniques. The system integrated predictive accuracy from three classifiers, namely naïve Bayes (NB), artificial neural network (ANN), and decision tree (DT) using a fuzzy logic-based fusion method. The proposed system, evaluated using NASA datasets, outperformed other techniques and aimed to achieve high-quality software with lower costs. A thorough comparative analysis of several classifiers was conducted in the context of software defect prediction [23]; the authors analyzed ten machine learning algorithms, including Decision Tree, Naive Bayes, K-Nearest Neighbor, Support Vector Machine, Random Forest, Extra Trees, Adaboost, Gradient Boosting, Bagging, and Multi-Layer Perceptron. The analysis was performed on benchmark NASA datasets from the PROMISE warehouse, specifically CM1, KC1, KC2, JM1, and PC1. The experimental results showed that the employed algorithms achieved higher average accuracy rates on the PC1 dataset. Among classifiers, the Random Forest learning models with the PCA approach exhibited boosted average performance across the datasets. Figure 4 illustrates the classification process.

In [24], the authors addressed the challenge of managing a large volume of software defect reports in software

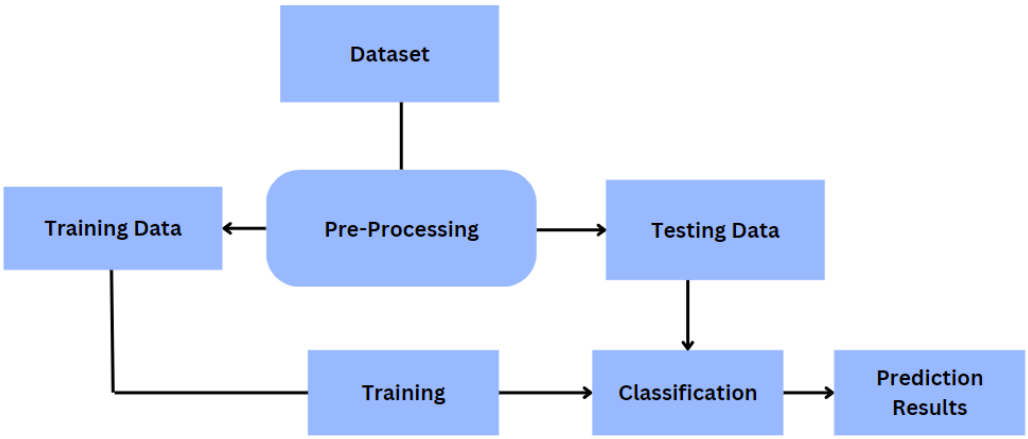

FIGURE 4. Classification process.

development and maintenance. They introduced a software defect prediction (SDP) model based on LASSO–SVM to improve prediction accuracy. The model combined feature selection using the minimum absolute value compression and selection method with the support vector machine algorithm. This approach significantly enhanced prediction accuracy, with simulation results showing an accuracy of 93.25% and 66.67%, a recall rate of 78.04%, and an f-measure of 72.72%. The proposed model outperformed traditional methods in terms of both accuracy and speed. In [25], researchers presented a cloud-based framework for real-time software defect prediction, comparing four back-propagation training algorithms. Bayesian regularization (BR) emerged as the most effective. The framework also included a fuzzy layer to select the best training function based on performance. Publicly available NASA datasets were used for evaluation, employing various measures. The results showed BR outperformed other training algorithms and widely used machine-learning techniques.

The researchers in [26] applied machine learning techniques to analyze the performance of different numbers of trees in the RF algorithm in the context of software defect prediction using the RAPIDMINER machine learning tool. They compared the performance of different numbers of trees in the RF algorithm. The results indicate that increasing the number of trees slightly improves accuracy, with a maximum accuracy of 99.59% and a minimum accuracy of 85.96%. The research also highlighted the effectiveness of the RF algorithm in software defect prediction, particularly when using around a hundred trees.

The author [27] evaluated various semi-supervised learning (SSL) techniques, mainly the extended random forest (extRF) technique, for predicting defective modules. The extRF technique extends the random forest approach and employs a weighted mixture of random trees for final predictions. The study concludes that SSL techniques, including active learning, can achieve improved prediction performance compared to traditional machine learning approaches. Researchers in [28] extensively studied the behaviour of various machine learning classification techniques for software defect prediction, including Naïve Bayes, Multi-Layer Perceptron, Radial Basis Function, Support Vector Machine, K Nearest Neighbor, kStar, One Rule, PART, Decision Tree, and Random Forest by employing NASA datasets. The performance of these techniques was evaluated using performance measures such as Precision, Recall, F-measure, accuracy, MCC, and ROC Area. The results indicated that accuracy and ROC may not be effective performance measures due to class imbalance issues, while precision, recall, F-Measure, and MCC are more reliable. The researchers in [29] introduced a novel approach that integrates genetic algorithms (GA) with support vector machine (SVM) classifiers and particle swarm algorithms for software fault prediction. The approach applies to large-scale (NASA MDP) and small-scale (Java open-source projects) datasets. The results demonstrate that integrating GA with SVM and particle swarm algorithms enhances the performance of software fault prediction, addressing limitations observed in previous studies. Authors in [30]introduced an algorithm based on support vector machines (SVM) and extreme learning machines (ELM) for software reliability prediction. They investigated the factors influencing prediction accuracy, such as using previous failure data and the appropriate type of failure information. They proposed a model for feature selection using ELM and SVM by addressing dataset imbalance issues through the resampling method and applying it to NASA Metrics datasets. Experimental results showed that the ELM-based reliability prediction model achieves higher accuracy, specificity, recall, precision, and F1-measure than SVM. The authors in [31] conducted research to highlight the use of ML methods, particularly SVM, for building software defect prediction models. Figure 5 illustrates defect prediction process using different classifiers. They evaluated the performance of SVM with different kernel functions on various datasets collected from software repositories. The researchers conducted 1520 experiments on 38 datasets.

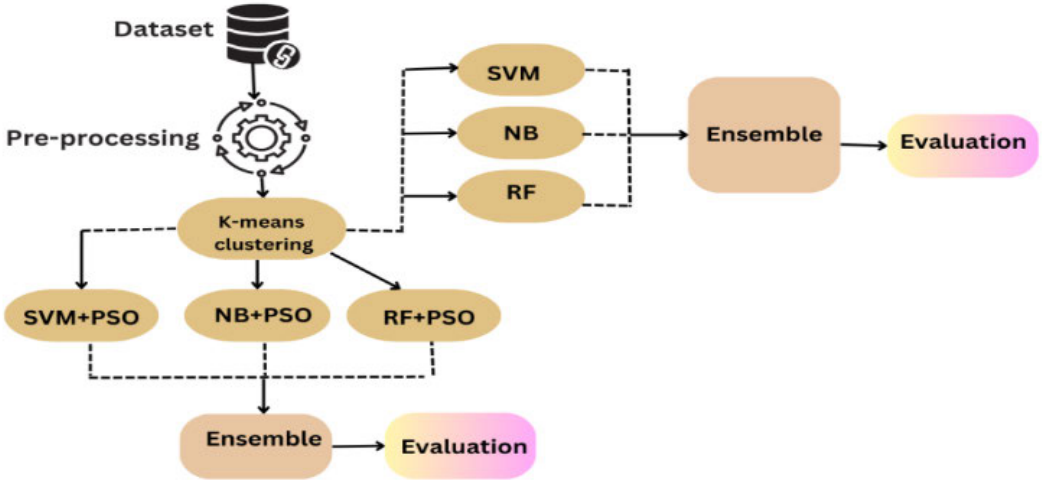

FIGURE 5. Defect prediction using different classifiers.

The performance of kernel functions was not significantly affected by the granularity of the data but did show differences in datasets with high imbalance ratios. RBF performed significantly better in highly imbalanced ratios, while other kernel functions like polynomial and linear performed well on moderately imbalanced datasets. The authors suggested using SVM with the RBF kernel for defect datasets due to its higher performance than other kernel functions. To improve software efficiency and reliability, the authors in [32] proposed an efficient and reliable framework for software defect prediction based on naive Bayes and linear regression. The framework consists of three main steps: data preprocessing (noise removal and normalization), feature extraction using correlation-based analysis, and applying machine learning models such as naive Bayes and linear regression. Results show that the proposed framework can reach an accuracy of 98.7% using the naive Bayes algorithm, which significantly reduces maintenance costs, lowers code complexity, and improves software quality by predicting defects early in the development process.

The authors in [33] conducted a study to investigate the impact of automated parameter optimization on defect prediction models using 18 datasets. The authors analyzed the classifiers' efficiency and stability, parameter transferability, computational cost, and ranking of the different classifying methods. The results reflected that automated parameter optimization improved the performance of defect prediction models, increasing the AUC (area under the curve) performance by up to forty percent. It was also observed that optimized classifiers were as stable as those with default settings, except for random forest classifiers. Grid-search optimization had a low computational cost, adding less than 30 minutes of additional computation time. Furthermore, some rarely-used techniques, like C5.0, outperform widely-used techniques after optimization.

Researchers in [34] addressed the challenges of data imbalance and high dimensionality in defect datasets. They employed several ML algorithms, such as Logistic Regression, Decision Trees, K-nearest neighbour, Support Vector Machines, and Ensemble Learning, along with feature extraction and selection techniques for classifying software modules as defect-prone or non-defect-prone. They proposed a model that utilized partial least squares (PLS) regression and RFE for dimension reduction and the synthetic minority oversampling technique (SMOTE) to handle imbalanced datasets [35]. The results show that XGBoost and Stacking Ensemble techniques yield the best results with a defect prediction accuracy above 0.9. Figure 4 represents defect prediction using different classifiers.

B. ENSEMBLE LEARNING

The authors in [36] proposed an intelligent system based on feature selection and ensemble machine learning techniques to predict defective modules in the software. A novel metric selection technique was introduced to select the most relevant features, and a three-step nested approach was employed for accurate prediction. The first step involves using a decision tree, support vector machines, and naïve Bayes to detect faulty modules. In the second step, the predictive accuracy of these techniques is integrated using ensemble methods such as bagging, voting, and stacking. Finally, fuzzy logic was applied to further combine the predictive accuracy of the ensemble techniques. The experiments were conducted on a fused software defect dataset, combining five NASA datasets, which demonstrated that the proposed system outperforms other advanced techniques, achieving an impressive accuracy rate of 92.08%.

Researchers in [37] developed a business failure prediction model for the US restaurant industry by using a majority voting ensemble method with a decision tree. They used experimental data from 1980 to 2017 and developed three models: an entire period model, an economic downturn model, and an economic expansion model. The models achieved prediction accuracies of 88.02%, 80.81%, and 87.02% respectively. Authors in [38] conducted thorough research to study

TABLE 2. Comprehensive summary of literature review.

Reference	Machine Learning Technique	Dataset Repository	Datasets	Performance Measures
[3]	The use of NN, DT, and ANN in the development of a cloud-based SDP system for integrating data and making decisions	NASA	CM1, MW1, PC1, PC3, PC4	Accuracy, Positive Prediction Value, Negative Prediction Value, Specificity, Sensitivity, False Positive Ratio, False Negative Ratio, Likelihood Ratio Positive, Likelihood Ratio Negative
[23]	A comparative analysis of machine learning-based SDP systems by analyzing ten supervised classification algorithms	PROMISE	CM1, KC1, KC2, JM1, PC1	Accuracy, Precision, Recall
[24]	LASSO – SVM-based model for software defect prediction using a reduced-dimension dataset	NASA	PC1	Precision, Recall, F- measure, Accuracy
[25]	Cloud-based framework for real-time SDP, comparing multiple back-propagation training algorithms	NASA	CM1, MC1, JM1, KC1, KC3, MC2, MW1, PC1, PC2, PC3, PC4, PC5	Precision, specificity, F-measure, recall, AUC, MSE, accuracy, and R2
[27]	Evaluation of various Semi-Supervised Learning (SSL) techniques, particularly the Extended Random Forest (extRF) technique for DSP	PROMISE	KC1, PC1, KC2, CM1	Precision, Recall, F-Measure
[28]	Performance analysis of supervised machine learning techniques for SDP	NASA	CM1, MC1,JM1, KC1, KC3, MC2, MW1, PC1, PC2,PC3, PC4, PC5	Precision, recall, F-measure, accuracy, MCC, and ROC area
[29]	Implementation of particle swarm algorithm with genetic algorithm and support vector machine classifier for SDP	NASA MDP, Java open-source projects	NASA (CM1, KC1, KC2, KC3, KC4, MC1, MC2, MW1, PC1, PC2, PC3, PC4, PC5) Java open-source projects (Poi2.5, Poi 3, Ivy 1.4, Ivy 2, log4j, jedit, synapse, lucene, xerces, xalan, camel, ant)	Accuracy, Standard division, recall, precision, specificity, error rate F-Measure
[30]	Feature selection-based classification for SDP using support vector machine and extreme learning machine algorithms	NASA	Various public NASA datasets	Precision, recall, accuracy, specificity, F1-Measure
[31]	SDP based on SVM using various kernel methods	AEEEM, NASA, SOFTLAB, ReLink, MORPH	38 public datasets	MCC, recall, Precision, AUC-PR
[32]	Implementation of naïve bayes classifier for SDP	PROMISE	PROMISE public dataset	Accuracy
[33]	Implementation of automated parameter tuning based on random forest and genetic algorithm for SDP	NASA, Proprietary, Apache, Eclipse	18 datasets	Precision, Recall, F-measure, G-mean, G-measure balance, MCC, specificity, false positive rate, AUC, Brier, LogLoss
[34]	Feature selection-based SDP Stacking ensemble classification	PROMISE	CM1, PC1, KC1, KC2	Accuracy, recall, precision, F-Measure
[35]	Ensemble classification based on Bagging, Voting, and Stacking for SDP	NASA	MW1, PC1, PC3, PC4, CM1	Misrata, Accuracy, Positive Prediction Value, Negative Prediction Value, Specificity, Sensitivity, False Positive Ratio, Likelihood Ratio Positive
[36]	SDP based on SVM using various kernel methods	AEEEM, NASA, SOFTLAB, ReLink, MORPH	38 public datasets	MCC, recall, Precision, AUC-PR
[37]	Majority voting ensemble for business failure prediction	dataset produced by Standard and Poor's Institutional Market Services	data from eating places categorized under the Standard Industrial Classification (SIC 5812) from 1980	Accuracy
[38]	A tree-based ensemble using stacking generalization	21 publicly available defect datasets	NASA, Eclipse	F-Measure, precision, recall, AUC, brier
[39]	Nested ensemble learning using voting as a main classifier along with bagging, boosting, and stacking as base classifiers for SDP	NASA	CM1, KC1, KC3, MC1, MW1, PC1, PC2, PC3, PC4, PC5	F-measure, MCC, Accuracy
[40]	multi-layer feed-forward neural networks using stacking as an ensemble technique for SDP	NASA	KC1, KC3, MC2, MW1, PC4 and PC5	Accuracy, Precision, Recall, F-measure, MCC, AUC

the effectiveness of seven Tree-based ensembles including bagging ensembles like Random Forest and Extra Trees, as well as boosting ensembles like AdaBoost, Gradient Boosting, Hist Gradient Boosting, XGBoost, and CatBoost.

They employed 11 publicly available NASA software defect datasets. The empirical results showed that the tree-based bagging ensembles, particularly random forest and extra trees, outperform the tree-based boosting ensembles.

In [39], a software defect prediction system was introduced by researchers. This system utilized a nested ensemble learning (EL) approach, with a Voting classifier as the main classifier and three base ensemble classifiers: bagging, boosting, and stacking. The accuracy achieved by this framework on two distinct NASA datasets was 83.46% and 79.65%. A software defect prediction model was proposed in a study conducted by researchers in [40]. This model employed multi-layer feed-forward neural networks in combination with stacking as an ensemble technique. Six different search methods were applied for feature selection to enhance the model's performance, with the multilayer perceptron serving as a subset evaluator. The achieved accuracies on NASA's datasets using the best-first search, greedy stepwise search, and GS methods were 80%, 75%, and 76%, respectively. Existing studies in software defect prediction have commonly employed individual classifiers, which, despite their utility, may suffer from limitations such as overfitting, lack of robustness, and biases inherent to specific algorithms.

Moreover, these standalone classifiers might not capture the diverse patterns present in complex software datasets, leading to suboptimal predictive performance [3], [28]. Although some studies have explored ensemble techniques, most have predominantly focused on homogeneous classifiers within their ensembles [23], [36]. In contrast, the proposed framework introduces a paradigm shift by integrating the predictive accuracy of heterogeneous classifiers–Random Forest (RF), Support Vector Machine (SVM), Naïve Bayes (NB), and Artificial Neural Network (ANN)–through a voting ensemble classification technique. This innovative combination addresses the shortcomings of both individual classifiers and conventional homogeneous ensemble approaches. By leveraging the strengths of diverse classifiers, the proposed model enhances interpretability, generalizability, and predictive accuracy, setting it apart as a more comprehensive and effective solution in software defect prediction. The summary of the literature review is shown in Table 2. It presents the ML techniques proposed for SDP, the source of datasets employed for research, the specific datasets used, and the performance measures implemented to analyze the results.

III. MATERIALS AND METHODS

The proposed research introduces an intelligent ensemble-based software defect prediction model. This model utilizes heterogeneous supervised ML classifiers for enhanced accuracy. The innovative approach aims to address challenges in predicting software defects efficiently. Individual base classifier outputs are consolidated through a voting ensemble model, strategically leveraging the strengths embedded in the proposed VESDP model. This ensemble approach enhances the predictive power of the model by synthesizing diverse classifier outputs, contributing to a more robust and accurate software defect prediction. An overview of the proposed model is shown in Figure 5. It shows that the proposed model VESDP model comprises two layers i.e. training and testing. There are three stages in the training layer: 1) data preprocessing 2) base classification 3) ensemble classification.

In the training layer, the dataset is preprocessed for classification. The predictive accuracy of base classifiers is then aggregated in an ensemble classifier, contributing to the development of the proposed ensemble-based model. The testing layer comprises one stage only namely prediction. This layer involves the defect prediction for unseen modules based on the trained model. The experiments have been performed using a Python tool that streamlined our data analysis process, allowing for efficient preprocessing, advanced statistical analysis, and accurate ML modeling enabling us to extract valuable insights from research data.

The following key steps were executed to identify defective modules in the software:

- In the first step, datasets comprising various software metrics were collected and reused [41].
- In the second step, preprocessing on the datasets was performed that further included three sub-activities, namely dataset splitting, cleaning, and normalization [42], [43]
- In the third step, the VESDP model was trained based on the diverse combination of base classifiers.
- In the fourth step, the base classifiers were integrated into an ensemble learning technique that aggregated the accuracy of base classifiers and produced unbiased and more accurate results.
- Finally, in the last step, the preprocessing technique was applied to the new modules, and the dataset was input to the trained model, which predicts defective modules.

The primary objective of the proposed approach is to predict the defective modules in software. The proposed VESDP model can be expressed by the following mapping.

$$Y = f(X) + \epsilon \tag{1}$$

where Y represents the prediction made by the model i.e., the module is defective or non-defective, X represents the module to be passed to the model for prediction purposes, and ϵ accounts for any deviation between the predicted output Y and the actual output. The key objective of this research is to minimize the amount of ϵ to make the defect prediction model more reliable and efficient.

$$X = x_1, x_2, x_3, \ldots\ldots x_n \tag{2}$$

where x1,x2,x3, . . .xn denote the attributes associated with the software module. This research aims to find this mapping through ensemble-based machine-learning techniques. The graphical representation of the proposed model having details of each step is shown in Figure 6.

A software module X consists of multiple attributes and can be represented as

Comprehensive details of each step involved in the training and testing layer are provided in the following sections.

A. DATASET COLLECTION

The first step in the training layer is the collection of historical software defect datasets. The selection of well-established benchmark datasets from NASA's MDP repository, including CM1, JM1, MC2, MW1, PC1, PC3, and PC4, was driven by a commitment to representativeness and comparability with existing research. Including these datasets aligns with industry standards, enhancing the generalizability of our proposed solution [41]. Each dataset corresponds to one software component, and each instance in the dataset represents one software module; also, one module consists of multiple software quality attributes, including LOC_COMMENT, LOC_TOTAL CALL_PAIRS, HALSTEAD_LENGTH, and HALSTEAD_CONSTANT, etc., that have been recorded during the development phase of SDLC. There are various dependent attributes and one independent attribute in each module. The dependent attributes (x1,x2,x3,. . .x) are used to make predictions, whereas the independent attribute also called the target variable, represents whether the module is defective (Y) or non-defective (N).

B. PREPROCESSING

Pre-processing is the second step of the training layer. This step further involves three sub-activities: 1) dataset splitting, 2) cleaning, and 3) normalization, which enhance the effectiveness of the proposed model. Dataset splitting is the first sub-activity of the pre-processing step. In this step, the used datasets are divided into two groups of training and testing with a ratio of 70:30 employing the class-based splitting rule [3]. The second sub-activity, cleaning, is pivotal for the model's robustness. Cleaning ensures the accuracy of predictions by removing inconsistent, inaccurate, or irrelevant data points. This step improves the quality and integrity of the data by reducing noise, handling missing values, ensuring consistency, and correcting errors within the dataset [44]. The cleaning activity is performed using the mean imputation method that replaced the missing values in the dataset, leading to better predictions.

The third sub-activity in the preprocessing step is normalization. Normalization is a widely used technique that scales and standardizes the input attributes of the dataset by equalizing feature scales within a range of 0 to 1 [42]. Normalization not only facilitates convergence but also contributes to the stability and efficiency of the machine-learning model. The process of equalizing feature scales eliminates the dominance of attributes with larger scales, preventing them from disproportionately influencing the model [43]. This, in turn, aids in achieving a balanced and unbiased learning process. Furthermore, normalization is instrumental in handling variations in data distribution, ensuring that the model performs consistently across different datasets [45]. Its role extends beyond numerical stability, encompassing the robustness and generalization capabilities of the proposed VESDP model.

C. CLASSIFICATION

Classification is the third step in the training layer that differentiates between defective and non-defective modules. It trains the models using labeled data and performs the identification of potentially faulty modules [46], [47]. In this research, four supervised machine learning classifiers of a heterogeneous nature have been implemented namely Random Forest (RF), Support Vector Machine (SVM), Naïve Bayes, and Artificial Neural Network (ANN).

The selection of Random Forest (RF), Support Vector Machine (SVM), Naïve Bayes, and Artificial Neural Network (ANN) as our base classifiers is rooted in their diverse and complementary strengths. RF excels in capturing complex relationships within data, particularly useful in software defect prediction scenarios [26]. SVM, with its ability to handle non-linear data through kernel functions, offers robust classification capabilities [29]. Naïve Bayes, relying on probabilistic principles, provides simplicity and efficiency in handling large datasets with conditional independence assumptions [48].

Lastly, inspired by neural networks, ANN demonstrates strong pattern recognition, which is essential for intricate software defect patterns [49]. The selection and optimization of these classifiers contribute to a well-rounded and resilient ensemble, enhancing the adaptability and accuracy of our proposed VESDP model. Initially, the model is developed using training data; based on the training data results, the classifiers have been optimized iteratively to achieve the highest accuracy.

It has been observed that RF performs best when split quality is measured using the Gini criterion and the depth of the tree is restricted to 10. SVM shows maximum accuracy when the kernel is set as poly and the complexity factor is set to 2. ANN performs best when hidden layers are set to 2, with 10 neurons in each layer. The rest of the parameters in RF, SVM, and ANN are used with default values. However, parameter tuning is not a primary concern in NB as it relies on the assumption that features are conditionally independent given the class variable; thus, it has been implemented with default parameters. The classification step ends by producing the optimized versions of all base classifiers.

D. ENSEMBLE MODELING

Ensemble modeling is the fourth step in the training layer. It refers to combining multiple individual models to make more accurate predictions or classifications. It is based on the concept of ''wisdom of the crowd,'' where combining the

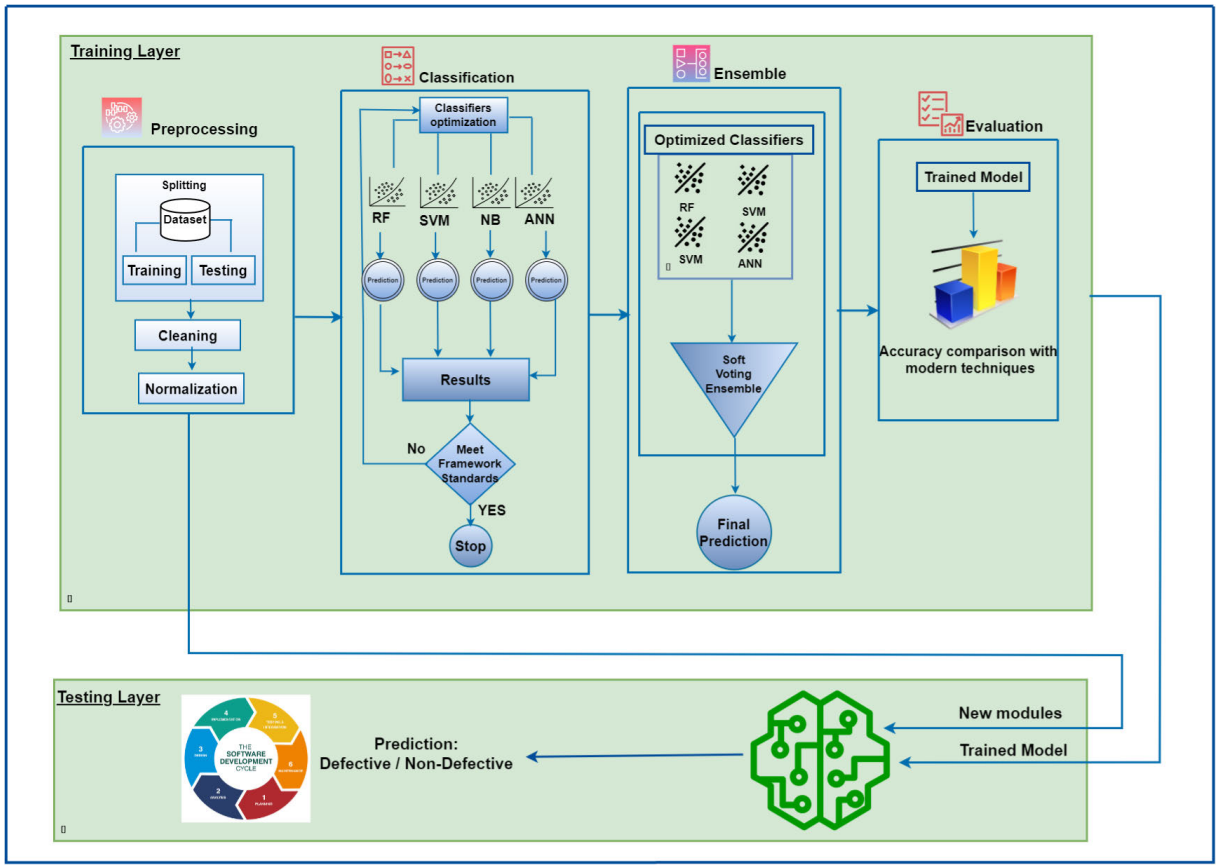

FIGURE 6. Proposed VESDP model.

predictions of multiple models often leads to better overall performance than relying on a single model [20], [50].

This research employs a voting ensemble model that boosts the accuracy and reliability of predictions [37]. The voting ensemble leverages the unique strengths of each base model, promoting a more robust and reliable prediction system. The diversity inherent in using multiple models mitigates biases and errors that might be present in any single model, fostering a clear understanding of software quality attributes. Furthermore, the ensemble's resilience to outliers and noise in the data adds an extra layer of robustness, ensuring more consistent and accurate predictions. Overfitting, a common challenge in machine learning, is also addressed as the voting ensemble method naturally reduces the risk of models fitting too closely to specific patterns in the training data.

The ensemble's ability to aggregate predictions leads to an overall improvement in accuracy, making it a valuable asset in the realm of software defect prediction, where precision and reliability are paramount for effective quality assurance in software development [20]. In the proposed model, the predictive accuracy of four heterogeneous base classifiers, including RF, SVM, NB, and ANN, is given as input to the voting ensemble model. The proposed VESDP model exhibits better performance on the voting ensemble as compared to base classifiers for the datasets used. The complete source code files along with the datasets employed in the framework; have been uploaded to GitHub Repository [51]. The pseudo-code of the main ensemble classifier is shown in Figure 7.

E. DEFECT PREDICTIONS USING THE VESDP MODEL

In this research, a voting ensemble-based software defect prediction model is proposed. The proposed model is applied in the testing layer, which comprises one step only, which is real-time predictions using unseen software modules. In this layer, unlabeled data is passed as input to the function $f(X)$ that attaches labels to the respective software modules. It is observed that the proposed model has a less error rate ϵ as compared to the modern techniques implemented for SDP. The output Y of the function $f(X)$ containing resultant predictions is sent back to the development team of SDLC that can debug the defective modules before passing it to the testing team; thus, saving time and effort for the quality assurance team and making it an economic process for the organization as well.

F. PERFORMANCE EVALUATIONL

In the above-mentioned formulas, $\alpha\ \lambda$ reflects the defective modules in the software which were correctly predicted as defective by the model; similarly, $\alpha\ \theta$ represents the

```
// Train the base classifiers
train_RF_classifier(training_data)
train_SVM_classifier(training_data)
train_NB_classifier(training_data)
train_ANN_classifier(training_data)

// Initialize class probabilities dictionary
class_probabilities = {class_label: 0 for class_label in class_labels}

// For each new data point
for data_point in new_data:
    // Make predictions using all base classifiers
    RF_prediction = RF_classifier.predict_proba(data_point)
    SVM_prediction = SVM_classifier.predict_proba(data_point)
    NB_prediction = NB_classifier.predict_proba(data_point)
    ANN_prediction = ANN_classifier.predict_proba(data_point)

    // Calculate the soft voting predictions
    for prediction, class_label in zip(RF_prediction, class_labels):
        class_probabilities[class_label] += prediction[1]

    for prediction, class_label in zip(SVM_prediction, class_labels):
        class_probabilities[class_label] += prediction[1]

    for prediction, class_label in zip(NB_prediction, class_labels):
        class_probabilities[class_label] += prediction[1]

    for prediction, class_label in zip(ANN_prediction, class_labels):
        class_probabilities[class_label] += prediction[1]

    // Identify the class with the highest probability or handle ties
    predicted_class = handle_ties(class_probabilities)

    // Output the predicted class
    output(predicted_class)
```

FIGURE 7. Pseudo-code of the main ensemble classifier.

non-defective modules in the software which were correctly predicted as non-defective by the model. $\beta\lambda$ and $\beta\theta$ Values indicate that there is a conflict between actual and predicted values. $\beta\lambda$ Shows that the module was non-defective but it was predicted as defective; on the other hand, $\beta\theta$ indicates that the module was defective but it was predicted as non-defective by the model.

$$Predicted\ positive\ value\ (PPV) = \frac{\alpha\lambda}{\alpha\lambda + \beta\lambda} \tag{3}$$

$$Predicted\ negative\ value\ (PNV) = \frac{\alpha\theta}{\alpha\theta + \beta\theta} \tag{4}$$

$$True\ positive\ rate\ (TPR) = \frac{\alpha\lambda}{\alpha\lambda + \beta\theta} \tag{5}$$

$$True\ negative\ rate\ (TNR) = \frac{\alpha\theta}{\alpha\theta + \beta\lambda} \tag{6}$$

$$Accuracy = \frac{\alpha\lambda + \alpha\theta}{\alpha\lambda + \alpha\theta + \beta\lambda + \beta\theta} \tag{7}$$

$$Misclassification\ rate(MR) = 1 - Accuracy \tag{8}$$

$$False\ positive\ rate(MR) = 1 - TNR \tag{9}$$

$$False\ negative\ rate(MR) = 1 - TPR \tag{10}$$

IV. RESULTS AND DISCUSSION

In this research, an intelligent voting ensemble-based software defect prediction model (VESDP) was implemented. To perform the experiments, seven publicly accessible NASA datasets (CM1, JM1, MC2, MW1, PC1, PC3, and PC4 were extracted from the MDP repository. In the preprocessing step, the datasets were subjected to three sub-activities, namely splitting, cleaning, and normalization. The splitting activity divides the datasets into two sub-sets, namely training, and

testing, with a ratio of 70:30 using the class-based splitting rule [53]. To train the model, initially, four heterogeneous supervised classification algorithms, including RF, SVM, NB, and ANN were implemented. These classifiers were optimized iteratively until each of them produced the highest possible accuracy for the used datasets. The predictive accuracy from individual classifiers is integrated using the voting ensemble technique, which further boosts the performance of the proposed model. To analyze the performance, eight widely used evaluation measures were implemented [22], [54]. All the performance measures have been derived using a confusion matrix that is drawn by implementing functions provided by Python [23], [55]. The results obtained from both training and testing datasets on each classifier are presented below.

Detailed results on the CM1 dataset are shown in Table 3. It is evident from the table that in the training phase, RF achieves 100% accuracy in both predicted positive and true positive rates. However, during the testing phase, RF maintains high accuracy at 85.86%, with a well-balanced trade-off between predicted positive values and true positive rates. This indicates RF's robustness in accurately predicting positive instances, even with previously unseen data. SVM performs well during both the training and testing phases, achieving an accuracy of 86.87% during testing. The classifier effectively differentiates between defective and non-defective modules. NB exhibits moderate accuracy during testing, with an accuracy of 80.81%. The model shows a balanced trade-off between false positive and false negative ratios, indicating its ability to make reasonably accurate predictions. ANN achieves an accuracy of 26.26% during testing. While the accuracy is comparatively lower, the model shows a balance in false positive and false negative ratios, suggesting its capability to classify defective and non-defective modules. VESDP model maintains an accuracy of 86.87% during testing. This underscores the effectiveness of the ensemble approach in combining diverse classifiers to enhance overall predictive performance, especially when dealing with unseen data. Detailed results on the CM1 dataset are shown in Table 3.

The performance measures attained by the CM1 test dataset, aside from accuracy, are shown in Figure 7.

The detailed results of the assessment of JM1 dataset are shown in Table 4. It reveals that in the training phase, RF classifier exhibits 100% accuracy in both predicted positive values and true positive rates. Whereas in the testing phase, RF maintains a high accuracy of 80.53%, demonstrating its capability to generalize well to unseen data. The classifier strikes a balance between predicted positive value and true positive rate, indicating its ability to make accurate predictions while minimizing false negatives. SVM achieves high accuracy during both training and testing, with a testing accuracy of 79.36%.

The model effectively differentiates between defective and non-defective modules, showcasing its robust performance. NB demonstrates moderate accuracy during testing, achieving 79.36%. The model maintains a balanced trade-off between false positive and false negative ratios, indicating its ability to make reliable predictions in the context of software defect prediction. ANN achieves an accuracy of 79.1% during testing. While the accuracy is lower compared to some models, ANN demonstrates a balanced performance concerning false positive and false negative ratios. VESDP model maintains a testing accuracy of 79.92%. This indicates that the ensemble approach effectively combines the strengths of individual classifiers to achieve robust performance in identifying defective and non-defective modules within the JM1 dataset.

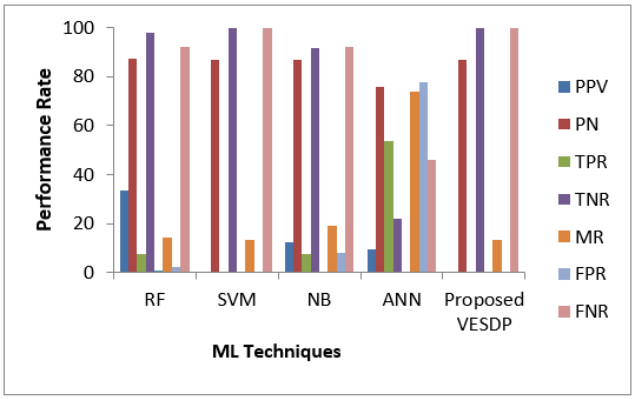

FIGURE 8. Performance measures on CM1 dataset.

The performance measures attained by JM1 testing datasets, aside from accuracy are shown in Figure 9.

The performance measures attained by the CM1 test dataset, aside from accuracy, are shown in Figure 10.

The detailed results of performance evaluation on the MC2 dataset are presented in Table 5. It presents that in the training phase, RF demonstrates 100% accuracy in both predicted positive values and true positive rates. While, in the testing phase, RF maintains a high accuracy of 63.61%, indicating its capability to generalize well to unseen data. The model shows a balanced trade-off between predicted positive value and true positive rate, suggesting its effectiveness in making accurate predictions while minimizing false negatives. SVM achieves a testing accuracy of 71.05%, demonstrating its effectiveness in distinguishing between defective and non-defective modules.

Detailed results on the MW1 dataset are presented in Table 6. It shows that during the training phase, RF exhibits 100% accuracy in both predicted positive and true positive rates. While in the testing phase, RF maintains a high accuracy of 88%, with a balance between predicted positive values and true positive rates. This suggests that RF continues to perform well in predicting positive instances even when confronted with new and unseen data. SVM demonstrates robust performance, achieving an accuracy of 86.67% during testing. The model maintains a high true negative rate, showcasing its ability to correctly identify non-defective modules and a low misclassification rate. NB performs

TABLE 3. Detailed results of classifiers on CM1 dataset.

ML Classifier	Dataset	Predicted positive value	Predicted negative value	True positive rate	True negative rate	Accuracy %	Misclassification Rate	False Positive Ratio	False Negative Ratio
RF	Training	1	1	1	1	100	0	0	0
	Testing	0.333	0.875	0.076	0.9767	85.86	14.14	0.023	0.923
SVM	Training	0	0.872	0	1	87.28	12.72	0	1
	Testing	0	0.868	0	1	86.87	13.13	0	1
NB	Training	0.322	0.903	0.344	0.894	82.46	17.54	0.105	0.655
	Testing	0.125	0.868	0.076	0.918	80.81	19.19	0.081	0.923
ANN	Training	0.092	0.805	0.482	0.311	33.33	66.67	0.688	0.517
	Testing	0.094	0.76	0.538	0.2209	26.26	73.74	0.779	0.461
Proposed VESDP	Training	0	0.872	0	1	87.28	12.72	0	1
	Testing	0	0.868	0	1	86.87	13.13	0	1

TABLE 4. Detailed results of classifiers on JM1 dataset.

ML Classifier	Dataset	Predicted positive value	Predicted negative value	True positive rate	True negative rate	Accuracy %	Misclassification Rate	False Positive Ratio	False Negative Ratio
RF	Training	1	0.861	1	1	87.31	12.69	0	0.608
	Testing	0.646	0.813	0.15	0.978	80.53	19.47	0.021	0.849
SVM	Training	0.666	0.792	0.005	0.999	79.18	20.82	0.0007	0.994
	Testing	1	0.793	0.12	1	79.36	20.64	0	0.987
NB	Training	0.489	0.81	0.148	0.959	79	21.0	0.04	0.851
	Testing	0.522	0.810	0.146	0.964	79.36	20.64	0.035	0.853
ANN	Training	0	0.791	0	1	79.13	20.87	0	1
	Testing	0	0.791	0	1	79.1	20.9	0	1
Proposed VESDP	Training	1	0.813	1	1	81.88	18.12	0	0.867
	Testing	0.771	0.799	0.055	.995	79.92	20.08	0.004	.944

TABLE 5. Detailed results of classifiers on MC2 dataset.

ML Classifier	Dataset	Predicted positive value	Predicted negative value	True positive rate	True negative rate	Accuracy %	Misclassification Rate	False Positive Ratio	False Negative Ratio
RF	Training	1	1	1	1	100	0	0	0
	Testing	0.461	0.72	0.461	0.72	63.61	36.84	0.28	0.538
SVM	Training	0	0.129	0.666	0.981	67.44	32.56	0.081	0.87
	Testing	1	0.694	0.15	1	71.05	28.95	0	0.846
NB	Training	0.75	0.728	0.387	0.927	73.26	26.74	0.072	0.619
	Testing	0.571	0.709	0.307	0.88	68.42	31.58	0.12	0.692
ANN	Training	0.09		0.064	0.636	43.02	56.98	0.363	0.935
	Testing	0.5	0.676	0.153	0.92	65.79	43.21	0.079	0.846
Proposed VESDP	Training	1	0.743	0.387	1	77.91	22.09	0	0.612
	Testing	0.6	0.696	0.23	0.92	68.42	31.58	0.079	0.769

reasonably well, with an accuracy of 78.67% during testing. The model's ability to maintain a balanced false positive ratio and false negative ratio suggests its reliability in distinguishing between defective and non-defective modules.

ANN faces challenges during testing, with an accuracy of 52%. The model exhibits a balanced false positive ratio and false negative ratio, indicating a moderate ability to classify both defective and non-defective modules. VESDP model excels during testing, achieving an accuracy of 89.33%. This shows the effectiveness of the ensemble approach in combining the strengths of individual classifiers to enhance overall predictive performance when applied to unseen data.

The performance measures, apart from accuracy, attained by the MW1 test dataset are represented in Figure 11.

Detailed results on the PC1 dataset are reflected in Table 7. It shows that in the training phase, the RF exhibits 100% accuracy in both predicted positive and true positive rates.

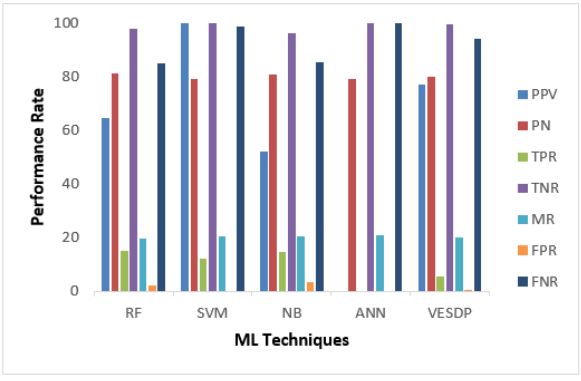

FIGURE 9. Performance measures on JM1 dataset.

While in the testing phase, it maintains a high accuracy of 93.63%, with a balanced trade-off between predicted positive values and true positive rates. This suggests RF's

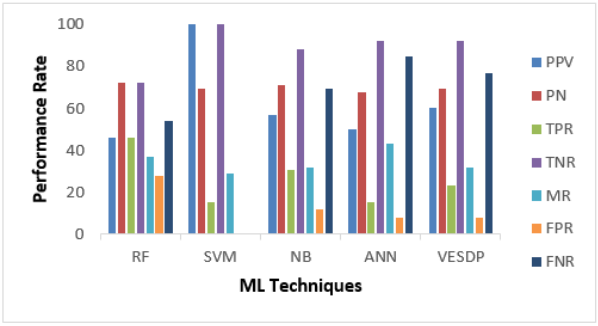

FIGURE 10. Performance measures on MC2 dataset.

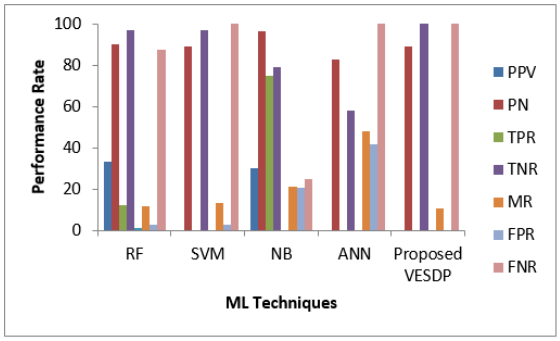

FIGURE 11. Performance measures on MW1 dataset.

effectiveness in accurately predicting positive instances even with new and unseen data, making it a robust choice for software defect prediction. SVM demonstrates strong performance during both the training and testing phases.

The classifier achieves an accuracy of 91.67% during testing, showcasing its ability to effectively distinguish between defective and non-defective modules. NB performs reasonably well, with an accuracy of 82.84% during testing. The classifier's balanced false positive ratio and false negative ratio suggest its reliability in classification, though there is room for improvement in certain scenarios. ANN achieves an accuracy of 85.29% during testing. The classifier shows a balanced false positive ratio and false negative ratio, indicating its moderate ability to classify both defective and non-defective modules. VESDP model excels during testing, achieving an accuracy of 92.16%. This exhibits the strength of the ensemble approach in combining diverse classifiers to enhance overall predictive performance, especially when applied to unseen data.

The performance measures achieved by the PC1 test dataset, excluding accuracy, are reflected in Figure 12. The detailed results achieved on the PC3 dataset by executing all the classifiers are presented in Table 8. It presents that in the training phase, the RF classifier achieves 100% accuracy in both predicted positive and true positive rates. While in the testing phase, it maintains high accuracy at 87.92%, with a well-balanced trade-off between predicted positive values and true positive rates. This suggests RF's robustness in

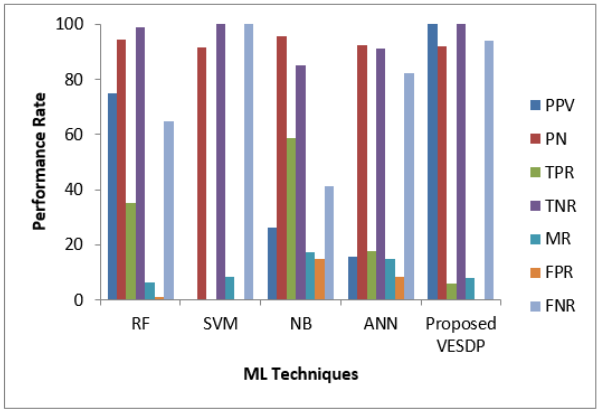

FIGURE 12. Performance measures on PC1 dataset.

accurately predicting positive instances, even with previously unseen data.

SVM demonstrates strong performance during both training and testing phases, achieving an accuracy of 87.66% during testing. The model effectively distinguishes between defective and non-defective modules. NB achieves an accuracy of 50% during testing, with a balanced false positive ratio and false negative ratio. This suggests that NB provides a baseline performance, and there is room for improvement in certain scenarios. ANN achieves an accuracy of 87.66% during testing, demonstrating balanced false positive and false negative ratios. This indicates moderate success in classifying both defective and non-defective modules. VESDP model excels during testing, achieving an accuracy of 87.97%. This demonstrates the strength of the ensemble approach in combining diverse classifiers to enhance overall predictive performance, particularly when applied to unseen data.

The performance measures achieved by PC3 test dataset, excluding accuracy are represented in Figure 13.

The results obtained from the PC4 dataset are shown in Table 9. It is clear from that table that in the training phase, RF shows 100% accuracy in both predicted positive and true positive rates. However, in the testing phase, it maintains high accuracy at 87.92%, with a well-balanced trade-off between predicted positive values and true positive rates. SVM demonstrates strong performance during both training and testing phases, achieving an accuracy of 87.66% during testing.

The classifier effectively distinguishes between defective and non-defective modules. NB achieves an accuracy of 50% during testing, with a balanced false positive ratio and false negative ratio. This suggests that NB provides a baseline performance, and there is room for improvement in certain scenarios. ANN achieves an accuracy of 87.66% during testing, demonstrating balanced false positive and false negative ratios. This indicates moderate success in classifying both defective and non-defective modules. VESDP model excels during testing, achieving an accuracy of 87.97%.

TABLE 6. Detailed results of classifiers on the MW1 dataset.

ML Classifier	Dataset	Predicted positive value	Predicted negative value	True positive rate	True negative rate	Accuracy %	Misclassification Rate	False Ratio	Positive	False Negative Ratio
RF	Training	1	1	1	1	100	0	0		0
	Testing	0.333	0.902	0.125	0.97	88	12	0.029		0.875
SVM	Training	1	0.908	0.058	1	90.86	9.14	0		0.941
	Testing	0	0.8904	0	0.9701	86.67	13.33	0.029		1
NB	Training	0.3103	0.945	0.529	0.873	84	16	0.126		0.4705
	Testing	0.3	0.963	0.75	0.79104	78.67	21.33	0.208		0.25
ANN	Training	0.041	0.862	0.176	0.556	52	48	0.443		0.823
	Testing	0	0.829	0	0.582	52	48	0.417		1
Proposed VESDP	Training	1	0.908	0.058	1	90.86	9.14	0		0.941
	Testing	0	0.893	0	1	89.33	10.67	0		1

TABLE 7. Detailed results of classifiers on the PC1 dataset.

ML Classifier	Dataset	Predicted positive value	Predicted negative value	True positive rate	True negative rate	Accuracy %	Misclassification Rate	False Ratio	Positive	False Negative Ratio
RF	Training	1	1	1	1	100	0	0		0
	Testing	0.75	0.943	0.352	0.989	93.63	6.37	0.0106		0.647
SVM	Training	0	0.92	0	1	92	8	0		1
	Testing	0	0.916	0	1	91.67	8.33	0		1
NB	Training	0.145	0.938	0.394	0.798	76.63	23.37	0.201		0.605
	Testing	0.263	0.957	0.588	0.8502	82.84	17.16	0.149		0.411
ANN	Training	0.139	0.925	0.157	0.915	85.47	14.53	0.084		0.842
	Testing	0.157	0.924	0.176	0.914	85.29	14.71	0.085		0.823
Proposed VESDP	Training	1	0.935	0.2105	1	93.68	6.32	0		0.789
	Testing	1	0.921	0.058	1	92.16	7.84	0		0.941

TABLE 8. Detailed results of classifiers on the PC3 dataset.

ML Classifier	Dataset	Predicted positive value	Predicted negative value	True positive rate	True negative rate	Accuracy %	Misclassification Rate	False Ratio	Positive	False Negative Ratio
RF	Training	1	1	1	1	100	0	0		0
	Testing	0.7	0.895	0.179	0.989	87.92	11.08	0.0108		0.8205
SVM	Training	0	0.876	0	1	87.65	12.35	0		1
	Testing	0	0.876	0	1	87.66	12.34	0		1
NB	Training	0.2	0.9807	0.934	0.473	53.05	46.95	0.526		0.065
	Testing	0.174	0.947	0.8205	0.454	50	50	0.545		0.179
ANN	Training	0	0.876	0	0.996	87.38	12.62	0.003		1
	Testing	0	0.876	0	1	87.66	12.34	0		1
Proposed VESDP	Training	1	0.8801	0.0329	1	88.06	11.94	0		0.967
	Testing	1	0.879	0.025	1	87.97	12.03	0		0.974

TABLE 9. Detailed results of classifiers on the PC4 dataset.

ML Classifier	Dataset	Predicted positive value	Predicted negative value	True positive rate	True negative rate	Accuracy %	Misclassification Rate	False Ratio	Positive	False Negative Ratio
RF	Training	1	1	1	1	100	0	0		0
	Testing	0.696	0.913	0.433	0.969	86.5	10.5	0.0304		0.566
SVM	Training	1	0.8655	0.0325	1	86.61	13.39	0		0.9674
	Testing	0	0.8608	0	1	86.09	13.91	0		1
NB	Training	0.428	0.878	0.1707	0.963	85.38	14.62	0.036		0.829
	Testing	0.48	0.884	0.226	0.9603	85.83	14.17	0.039		0.773
ANN	Training	0.129	0.859	0.186	0.797	71.32	28.68	0.202		0.813
	Testing	0.118	0.855	0.169	0.795	70.87	29.13	0.204		0.8301
Proposed VESDP	Training	1	0.881	0.162	1	88.41	11.59	0		0.837
	Testing	0.833	0.872	0.094	0.996	87.14	12.86	0.003		0.905

This highlights the strength of the ensemble approach in combining diverse classifiers to enhance overall predictive performance, particularly when applied to unseen data. The detailed results obtained from the PC4 dataset are displayed in Table 9.

The performance measures attained by the PC4 test dataset, except accuracy, are displayed in Figure14.

A. PERFORMANCE COMPARISON

In this section, the proposed voting ensemble-based software defect prediction (VESDP) model is compared based on accuracy with modern research that has implemented state-of-the-art techniques for software defect prediction. The comparison is based on twenty published studies in the last 5 years. Out of twenty studies, ten researchers worked on

TABLE 10. Comparison of VESDP with state-of-the-art techniques.

Datasets	CM1	JM1	MC2	MW1	PC1	PC3	PC4
[21]	83	78	68		91	84	84
[56]	84.1			83.6			
[57]	82.28	78.08	86.52	91.15	85.79		
[28]	77.55	73.96	64.86	82.66		82.59	86.08
[58]	83	77					
[59]	84						
[52]	83.79			83.79			
[60]	84.94	77.93					
[61]		31.79		83.02			
[22]	79.59	62.78	62.16	77.33	89.65	75.94	74.8
[55]		49.43					
[11]			67.29	86.89			
[19]							86.61
[47]			67.56		89.7	87.34	
[63]			68.32	60.05		73.7	86.9
[29]			67.09				
[6]					92		
[65]					91.9	84.59	
[14]						81.92	
[54]	81.79				89.79		
VESDP	86.87	79.12	68.42	89.33	92.16	87.97	87.14

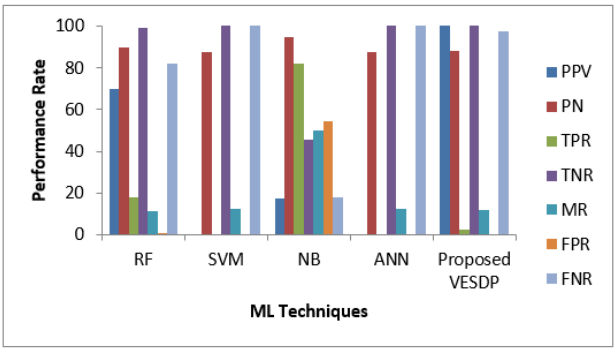

FIGURE 13. Performance measures on PC3 dataset.

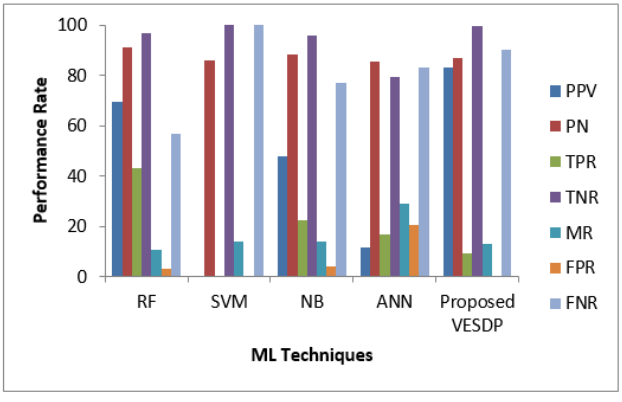

FIGURE 14. Performance measures on PC4 dataset.

the CM1 dataset, eight on MW1, JM1, and PC3 datasets, seven on MC2 and PC1 datasets, and five on the PC4 dataset. It is clear from the results that the integration of

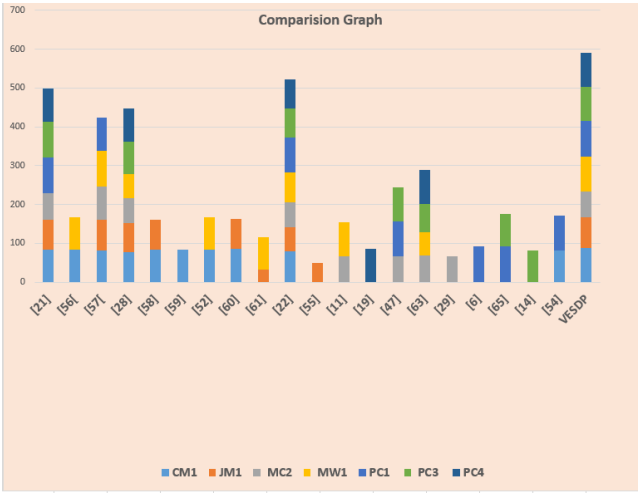

FIGURE 15. Comparison graph.

heterogeneous base classifiers into ensemble classification boosts the accuracy of the software defect prediction process. The accuracy comparison of the proposed VESDP model with modern techniques is shown in Table 10. Figure 15 illustrates performance comparison.

V. THREATS TO VALIDITY

Validity threats refer to factors or issues that may undermine the accuracy, credibility, or generalizability of the findings in a research paper. These threats can arise at different stages of the research process and can affect the validity of the study's

conclusions [66]. Some most crucial validity threats are listed below:

A. INTERNAL VALIDITY

This type of validity assesses the adequacy of the chosen prediction techniques for the specific datasets employed in the research or for other datasets used to tackle the same problem [67].

In this research, four supervised classification algorithms were used to implement the proposed VESDP model namely RF, SVM, NB, and ANN based on the diversity in their computation mechanism and performance. In the future, researchers can implement clustering algorithms along with feature selection techniques to analyze the performance of software defect prediction models.

B. EXTERNAL VALIDITY

This form of validity examines whether the proposed solution is equally effective when applied to other datasets associated with the same problem domain [68]. In this research, seven benchmark datasets namely CM1, MW1, PC1, PC3, and PC4 from NASA's defect repository were employed to implement the proposed VESDP model. Hence, the conclusion of this research cannot be generalized to other defect datasets having different attributes. However, the preprocessing steps including dataset splitting, cleaning, and normalization along with parameter optimization in the classification step can be implemented by other researchers in their studies.

C. CONSTRUCT VALIDITY

This validity is concerned with the performance measures used to evaluate the proposed model [69]. In this research, eight performance measures including predicted positive value, predicted negative value, true positive rate, true negative rate, accuracy, misclassification rate, false positive ratio, and false negative ratio have been implemented to evaluate the performance of the proposed VESDP model. However, only an accuracy measure is used to compare the performance of the VESDP model with state-of-the-art techniques.

D. CONCLUSION VALIDITY

This validity refers to the extent to which the conclusion drawn from a study accurately represents the true relationships or effects observed in the proposed model [68]. In this research, the conclusion is drawn based on the accuracy comparison with state-of-the-art techniques. This assessment shows that the proposed model has better results as compared to modern research.

VI. CONCLUSION

Software defect prediction aims to identify faulty modules before the testing phase, enabling a focus on testing only those modules that are likely to have defects. An efficient defect prediction model can reduce software development costs by minimizing the resources dedicated to quality assurance activities during testing. In this research, an intelligent ensemble-based model for software defect prediction was proposed. The model was implemented using benchmark datasets extracted from the NASA defect repository. The proposed model integrated the predictive accuracy of four heterogeneous supervised classifiers using the voting ensemble classification technique. For statistical analysis, eight performance measures were implemented. To prove the effectiveness of the strategy adopted in the proposed model, a comparative analysis was conducted with state-of-the-art techniques. The proposed VESDP model outperformed modern research and proved its efficiency for the software defect prediction process.

VII. LIMITATION OF PROPOSED MODEL

The training data has a significant impact on the effectiveness of any machine-learning model, including ensemble-based models. The training dataset's disparity, missing data, or noise may have a detrimental effect on the model's ability to predict the future. The requirements, development processes, and code used in software development are continually changing. An ensemble-based model trained on historical data may struggle to adapt when faced with unexpected changes in project dynamics or new development paradigms.The model looks for trends in historical data to produce forecasts. If the current task differs greatly from the projects in the training dataset, the model's performance can suffer.

VIII. FUTURE WORK

For future work, it is recommended to explore advanced feature selection techniques using a genetic algorithm or bat search algorithm to enhance the robustness of the proposed model in software defect prediction. Additionally, incorporating nested ensemble techniques using bagging, boosting, stacking, etc. could further improve the reliability of predictions. These enhancements would contribute to the continuous evolution of defect prediction models, advancing their applicability in real-world software development scenarios.

IX. FUNDING

Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2024R136), Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia.

REFERENCES

[1] Z. M. Zain, S. Sakri, and N. H. A. Ismail, "Application of deep learning in software defect prediction: Systematic literature review and meta-analysis," *Inf. Softw. Technol.*, vol. 158, Jun. 2023, Art. no. 107175, doi: 10.1016/j.infsof.2023.107175.

[2] M. Unterkalmsteiner et al., "Software startups—A research agenda," 2023, *arXiv:2308.12816*.

[3] S. Aftab, S. Abbas, T. M. Ghazal, M. Ahmad, H. A. Hamadi, C. Y. Yeun, and M. A. Khan, "A cloud-based software defect prediction system using data and decision-level machine learning fusion," *Mathematics*, vol. 11, no. 3, p. 632, Jan. 2023, doi: 10.3390/math11030632.

[4] S. Goyal, "Heterogeneous stacked ensemble classifier for software defect prediction," in *Proc. 6th Int. Conf. Parallel, Distrib. Grid Comput. (PDGC)*, Waknaghat, India, Nov. 2020, pp. 126–130, doi: 10.1109/PDGC50313.2020.9315754.

[5] S. Mehta and K. S. Patnaik, "Stacking based ensemble learning for improved software defect prediction," in *Proc. 5th Int. Conf. Microelectron., Comput. Commun. Syst.*, vol. 748, 2021, pp. 167–178.

[6] M. Shafiq, F. H. Alghamedy, N. Jamal, T. Kamal, Y. I. Daradkeh, and M. Shabaz, "Retracted: Scientific programming using optimized machine learning techniques for software fault prediction to improve software quality," *IET Softw.*, vol. 17, no. 4, pp. 694–704, Jan. 2023, doi: 10.1049/sfw2.12091.

[7] Y. Tang, Q. Dai, M. Yang, T. Du, and L. Chen, "Software defect prediction ensemble learning algorithm based on adaptive variable sparrow search algorithm," *Int. J. Mach. Learn. Cybern.*, vol. 14, no. 6, pp. 1967–1987, Jan. 2023, doi: 10.1007/s13042-022-01740-2.

[8] S. Goyal, "3PcGE: 3-parent child-based genetic evolution for software defect prediction," *Innov. Syst. Softw. Eng.*, vol. 19, no. 2, pp. 197–216, Jun. 2023, doi: 10.1007/s11334-021-00427-1.

[9] J. Liu, J. Ai, M. Lu, J. Wang, and H. Shi, "Semantic feature learning for software defect prediction from source code and external knowledge," *J. Syst. Softw.*, vol. 204, Oct. 2023, Art. no. 111753, doi: 10.1016/j.jss.2023.111753.

[10] A. K. Gangwar and S. Kumar, "Concept drift in software defect prediction: A method for detecting and handling the drift," *ACM Trans. Internet Technol.*, vol. 23, no. 2, pp. 1–28, May 2023, doi: 10.1145/3589342.

[11] M. S. Alkhasawneh, "Software defect prediction through neural network and feature selections," *Appl. Comput. Intell. Soft Comput.*, vol. 2022, pp. 1–16, Sep. 2022, doi: 10.1155/2022/2581832.

[12] T. F. Husin and M. R. Pribadi, "Implementation of LSSVM in classification of software defect prediction data with feature selection," in *Proc. 9th Int. Conf. Electr. Eng., Comput. Sci. Informat. (EECSI)*, Jakarta, Indonesia, Oct. 2022, pp. 126–131, doi: 10.23919/EECSI56542.2022.9946611.

[13] J. A. Richards, "Supervised classification techniques," in *Remote Sensing Digital Image Analysis*. Cham, Switzerland: Springer, 2022, pp. 263–367.

[14] B. J. Odejide, A. O. Bajeh, A. O. Balogun, Z. O. Alanamu, K. S. Adewole, A. G. Akintola, and S. A. Salihu, "An empirical study on data sampling methods in addressing class imbalance problem in software defect prediction," in *Proc. Comput. Sci. Online Conf.* Cham, Switzerland: Springer, Apr. 2022, pp. 594–610.

[15] X. Wu and J. Wang, "Application of bagging, boosting and stacking ensemble and EasyEnsemble methods for landslide susceptibility mapping in the three Gorges reservoir area of China," *Int. J. Environ. Res. Public Health*, vol. 20, no. 6, p. 4977, Mar. 2023, doi: 10.3390/ijerph20064977.

[16] F. Jiang, X. Yu, D. Gong, and J. Du, "A random approximate reduct-based ensemble learning approach and its application in software defect prediction," *Inf. Sci.*, vol. 609, pp. 1147–1168, Sep. 2022, doi: 10.1016/j.ins.2022.07.130.

[17] H. Chen, X.-Y. Jing, Y. Zhou, B. Li, and B. Xu, "Aligned metric representation based balanced multiset ensemble learning for heterogeneous defect prediction," *Inf. Softw. Technol.*, vol. 147, Jul. 2022, Art. no. 106892, doi: 10.1016/j.infsof.2022.106892.

[18] A. O. Balogun, A. O. Bajeh, V. A. Orie, and A. W. Yusuf-Asaju, "Software defect prediction using ensemble learning: An ANP based evaluation method," *FUOYE J. Eng. Technol.*, vol. 3, no. 2, pp. 50–55, Sep. 2018, doi: 10.46792/fuoyejet.v3i2.200.

[19] A. O. Balogun, F. B. Lafenwa-Balogun, H. A. Mojeed, V. E. Adeyemo, O. N. Akande, A. G. Akintola, A. O. Bajeh, and F. E. Usman-Hamza, "SMOTE-based homogeneous ensemble methods for software defect prediction," in *Computational Science and Its Applications—ICCSA 2020*, vol. 12254, O. Gervasi, B. Murgante, S. Misra, C. Garau, I. B. D. Taniar, B. O. Apduhan, A. M. A. C. Rocha, E. Tarantino, C. M. Torre, and Y. Karaca, Eds. Cham, Switzerland: Springer, 2020, pp. 615–631.

[20] R. J. Jacob, R. J. Kamat, N. M. Sahithya, S. S. John, and S. P. Shankar, "Voting based ensemble classification for software defect prediction," in *Proc. IEEE Mysore Sub Sect. Int. Conf. (MysuruCon)*, Hassan, India, Oct. 2021, pp. 358–365, doi: 10.1109/MysuruCon52639.2021.9641713.

[21] A. Alsaeedi and M. Z. Khan, "Software defect prediction using supervised machine learning and ensemble techniques: A comparative study," *J. Softw. Eng. Appl.*, vol. 12, no. 5, pp. 85–100, 2019, doi: 10.4236/jsea.2019.125007.

[22] A. Iqbal and S. Aftab, "A classification framework for software defect prediction using multi-filter feature selection technique and MLP," *Int. J. Mod. Educ. Comput. Sci.*, vol. 12, no. 1, pp. 18–25, Feb. 2020, doi: 10.5815/ijmecs.2020.01.03.

[23] M. Cetiner and O. K. Sahingoz, "A comparative analysis for machine learning based software defect prediction systems," in *Proc. 11th Int. Conf. Comput., Commun. Netw. Technol. (ICCCNT)*, Kharagpur, India, Jul. 2020, pp. 1–7, doi: 10.1109/ICCCNT49239.2020.9225352.

[24] K. Wang, L. Liu, C. Yuan, and Z. Wang, "Software defect prediction model based on LASSO–SVM," *Neural Comput. Appl.*, vol. 33, no. 14, pp. 8249–8259, Jul. 2021, doi: 10.1007/s00521-020-04960-1.

[25] M. S. Daoud, S. Aftab, M. Ahmad, M. A. Khan, A. Iqbal, S. Abbas, M. Iqbal, and B. Ihnaini, "Machine learning empowered software defect prediction system," *Intell. Autom. Soft Comput.*, vol. 31, no. 2, pp. 1287–1300, 2022, doi: 10.32604/iasc.2022.020362.

[26] Y. N. Soe, P. I. Santosa, and R. Hartanto, "Software defect prediction using random forest algorithm," in *Proc. 12th South East Asian Technical Univ. Consortium*, Yogyakarta, Indonesia, Mar. 2018, pp. 1–5, doi: 10.1109/SEATUC.2018.8788881.

[27] F. H. Alshammari, "Software defect prediction and analysis using enhanced random forest (extRF) technique: A business process management and improvement concept in IoT-based application processing environment," *Mobile Inf. Syst.*, vol. 2022, pp. 1–11, Sep. 2022, doi: 10.1155/2022/2522202.

[28] A. Iqbal, S. Aftab, U. Ali, Z. Nawaz, L. Sana, M. Ahmad, and A. Husen, "Performance analysis of machine learning techniques on software defect prediction using NASA datasets," *Int. J. Adv. Comput. Sci. Appl.*, vol. 10, no. 5, 2019, doi: 10.14569/IJACSA.2019.0100538.

[29] H. Alsghaier and M. Akour, "Software fault prediction using particle swarm algorithm with genetic algorithm and support vector machine classifier," *Softw., Pract. Exper.*, vol. 50, no. 4, pp. 407–427, Apr. 2020, doi: 10.1002/spe.2784.

[30] S. K. Rath, M. Sahu, S. P. Das, S. K. Bisoy, and M. Sain, "A comparative analysis of SVM and ELM classification on software reliability prediction model," *Electronics*, vol. 11, no. 17, p. 2707, Aug. 2022, doi: 10.3390/electronics11172707.

[31] M. Azzeh, Y. Elsheikh, A. B. Nassif, and L. Angelis, "Examining the performance of kernel methods for software defect prediction based on support vector machine," *Sci. Comput. Program.*, vol. 226, Mar. 2023, Art. no. 102916, doi: 10.1016/j.scico.2022.102916.

[32] A. Rahim, Z. Hayat, M. Abbas, A. Rahim, and M. A. Rahim, "Software defect prediction with Naïve Bayes classifier," in *Proc. Int. Bhurban Conf. Appl. Sci. Technol. (IBCAST)*, Islamabad, Pakistan, Jan. 2021, pp. 293–297, doi: 10.1109/ibcast51254.2021.9393250.

[33] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "The impact of automated parameter optimization on defect prediction models," *IEEE Trans. Softw. Eng.*, vol. 45, no. 7, pp. 683–711, Jul. 2019, doi: 10.1109/TSE.2018.2794977.

[34] S. Mehta and K. S. Patnaik, "Improved prediction of software defects using ensemble machine learning techniques," *Neural Comput. Appl.*, vol. 33, no. 16, pp. 10551–10562, Aug. 2021, doi: 10.1007/s00521-021-05811-3.

[35] A. Khalid, G. Badshah, N. Ayub, M. Shiraz, and M. Ghouse, "Software defect prediction analysis using machine learning techniques," *Sustainability*, vol. 15, no. 6, p. 5517, Mar. 2023, doi: 10.3390/su15065517.

[36] S. Abbas, S. Aftab, M. A. Khan, T. M. Ghazal, H. A. Hamadi, and C. Y. Yeun, "Data and ensemble machine learning fusion based intelligent software defect prediction system," *Comput., Mater. Continua*, vol. 75, no. 3, pp. 6083–6100, 2023, doi: 10.32604/cmc.2023.037933.

[37] S. Y. Kim and A. Upneja, "Majority voting ensemble with a decision trees for business failure prediction during economic downturns," *J. Innov. Knowl.*, vol. 6, no. 2, pp. 112–123, Apr. 2021, doi: 10.1016/j.jik.2021.01.001.

[38] A. Alazba and H. Aljamaan, "Software defect prediction using stacking generalization of optimized tree-based ensembles," *Appl. Sci.*, vol. 12, no. 9, p. 4577, Apr. 2022, doi: 10.3390/app12094577.

[39] M. A. Javed. (2021). *A Framework for Software Defect Prediction Using Nested-Ensemble Learning and Feature Selection Techniques*. [Online]. Available: https://vspace.vu.edu.pk/detail.aspx?id=592

[40] F. Matloob. (2020). *Software Defect Prediction Model Using Multi-Layer Feed Forward Neural Networks*. [Online]. Available: https://vspace.vu.edu.pk/detail.aspx?id=342

[41] M. Shepperd, Q. Song, Z. Sun, and C. Mair, "Data quality: Some comments on the NASA software defect datasets," *IEEE Trans. Softw. Eng.*, vol. 39, no. 9, pp. 1208–1215, Sep. 2013, doi: 10.1109/TSE.2013.11.

[42] J. Shi, X. Li, L. Li, C. Ouyang, and C. Xu, "An efficient deep learning-based troposphere ZTD dataset generation method for massive GNSS CORS stations," *IEEE Trans. Geosci. Remote Sens.*, 2023.

[43] W. Du, C. Wu, H. Yu, Q. Kong, Y. Xu, and W. Zhang, "Determination of multicomponents in *Rubi Fructus* by near-infrared spectroscopy technique," *Int. J. Anal. Chem.*, vol. 2023, pp. 1–9, Nov. 2023, doi: 10.1155/2023/5575944.

[44] P. Suresh Kumar, H. S. Behera, J. Nayak, and B. Naik, "Bootstrap aggregation ensemble learning-based reliable approach for software defect prediction by using characterized code feature," *Innov. Syst. Softw. Eng.*, vol. 17, no. 4, pp. 355–379, Dec. 2021, doi: 10.1007/s11334-021-00399-2.

[45] H. Tong, S. Wang, and G. Li, "Credibility based imbalance boosting method for software defect proneness prediction," *Appl. Sci.*, vol. 10, no. 22, p. 8059, Nov. 2020, doi: 10.3390/app10228059.

[46] H. Alsawalqah, N. Hijazi, M. Eshtay, H. Faris, A. A. Radaideh, I. Aljarah, and Y. Alshamaileh, "Software defect prediction using heterogeneous ensemble classification based on segmented patterns," *Appl. Sci.*, vol. 10, no. 5, p. 1745, Mar. 2020, doi: 10.3390/app10051745.

[47] A. Iqbal, S. Aftab, I. Ullah, M. S. Bashir, and M. A. Saeed, "A feature selection based ensemble classification framework for software defect prediction," *Int. J. Modern Educ. Comput. Sci.*, vol. 11, no. 9, pp. 54–64, Sep. 2019, doi: 10.5815/ijmecs.2019.09.06.

[48] F. M. Tua and W. Danar Sunindyo, "Software defect prediction using software metrics with Naïve Bayes and rule mining association methods," in *Proc. 5th Int. Conf. Sci. Technol. (ICST)*, Yogyakarta, Indonesia, Jul. 2019, pp. 1–5, doi: 10.1109/icst47872.2019.9166448.

[49] S. I. Ayon, "Neural network based software defect prediction using genetic algorithm and particle swarm optimization," in *Proc. 1st Int. Conf. Adv. Sci., Eng. Robot. Technol. (ICASERT)*, Dhaka, Bangladesh, May 2019, pp. 1–4, doi: 10.1109/ICASERT.2019.8934642.

[50] T. Zhang, Y. Yu, X. Mao, Y. Lu, Z. Li, and H. Wang, "FENSE: A feature-based ensemble modeling approach to cross-project just-in-time defect prediction," *Empirical Softw. Eng.*, vol. 27, no. 7, p. 162, Dec. 2022, doi: 10.1007/s10664-022-10185-8.

[51] [Online]. Available: https://github.com/misbah-here/VESDP_Repository

[52] A. O. Balogun, S. Basri, S. A. Jadid, S. Mahamad, M. A. Al-momani, A. O. Bajeh, and A. K. Alazzawi, "Search-based wrapper feature selection methods in software defect prediction: An empirical analysis," in *Intelligent Algorithms in Software Engineering*, vol. 1224, R. Silhavy, Ed. Cham, Switzerland: Springer International Publishing, 2020, pp. 492–503.

[53] I. Kaur and A. Kaur, "Comparative analysis of software fault prediction using various categories of classifiers," *Int. J. Syst. Assurance Eng. Manage.*, vol. 12, no. 3, pp. 520–535, Jun. 2021, doi: 10.1007/s13198-021-01110-1.

[54] B. Mumtaz, S. Kanwal, S. Alamri, and F. Khan, "Feature selection using artificial immune network: An approach for software defect prediction," *Intell. Autom. Soft Comput.*, vol. 29, no. 3, pp. 669–684, 2021, doi: 10.32604/iasc.2021.018405.

[55] S. Goyal, "Handling class-imbalance with KNN (neighbourhood) under-sampling for software defect prediction," *Artif. Intell. Rev.*, vol. 55, no. 3, pp. 2023–2064, Mar. 2022, doi: 10.1007/s10462-021-10044-w.

[56] A. O. Balogun, S. Basri, S. J. Abdulkadir, and A. S. Hashim, "Performance analysis of feature selection methods in software defect prediction: A search method approach," *Appl. Sci.*, vol. 9, no. 13, p. 2764, Jul. 2019.

[57] H. Aljamaan and A. Alazba, "Software defect prediction using tree-based ensembles," in *Proc. 16th ACM Int. Conf. Predictive Models Data Anal. Softw. Eng.*, New York, NY, USA, Nov. 2020, pp. 1–10, doi: 10.1145/3416508.3417114.

[58] M. Azam, M. Nouman, and A. R. Gill, "Comparative analysis of machine learning technique to improve software defect prediction," *KIET J. Comput. Inf. Sci.*, vol. 5, no. 2, pp. 1–11, Jul. 2022, doi: 10.51153/kjcis.v5i2.96.

[59] S. Goyal and P. K. Bhatia, "Comparison of machine learning techniques for software quality prediction," *Int. J. Knowl. Syst. Sci.*, vol. 11, no. 2, pp. 20–40, Apr. 2020, doi: 10.4018/IJKSS.2020040102.

[60] U. S. Bhutamapuram and R. Sadam, "With-in-project defect prediction using bootstrap aggregation based diverse ensemble learning technique," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 10, pp. 8675–8691, Nov. 2022, doi: 10.1016/j.jksuci.2021.09.010.

[61] A. Balogun, R. O. Oladele, H. A. Mojeed, and B. Amin-Balogun, "Performance analysis of selected clustering techniques for software defects prediction," *Afr. J. Comput. ICT*, vol. 12, no. 2, pp. 30–42, 2019.

[62] M. A. Javed, "A framework for software defect prediction using nested-ensemble learning and feature selection techniques," M.S. thesis, Virtual Univ. Pakistan, Lahore, Pakistan, 2021. [Online]. Available: https://vspace.vu.edu.pk/details.aspx?id=592

[63] A. O. Balogun, S. Basri, L. F. Capretz, S. Mahamad, A. A. Imam, M. A. Almomani, V. E. Adeyemo, A. K. Alazzawi, A. O. Bajeh, and G. Kumar, "Software defect prediction using wrapper feature selection based on dynamic re-ranking strategy," *Symmetry*, vol. 13, no. 11, p. 2166, Nov. 2021, doi: 10.3390/sym13112166.

[64] S. Singh and T. U. Haider, "Selection of best feature reduction method for module-based software defect prediction," *J. Phys., Conf. Ser.*, vol. 2273, no. 1, May 2022, Art. no. 012002, doi: 10.1088/1742-6596/2273/1/012002.

[65] S. Amin. (2019). *Software Defect Prediction via Machine Learning Classifiers*. [Online]. Available: https://vspace.vu.edu.pk/detail.aspx?id=378

[66] F. Yucalar, A. Ozcift, E. Borandag, and D. Kilinc, "Multiple-classifiers in software quality engineering: Combining predictors to improve software fault prediction ability," *Eng. Sci. Technol., Int. J.*, vol. 23, no. 4, pp. 938–950, Aug. 2020, doi: 10.1016/j.jestch.2019.10.005.

[67] U. Sharma B and R. Sadam, "Towards developing and analysing metric-based software defect severity prediction model," 2022, *arXiv:2210.04665*.

[68] A. Abdu, Z. Zhai, R. Algabri, H. A. Abdo, K. Hamad, and M. A. Al-antari, "Deep learning-based software defect prediction via semantic key features of source code—Systematic survey," *Mathematics*, vol. 10, no. 17, p. 3120, Aug. 2022.

[69] Z. Xu, J. Liu, X. Luo, Z. Yang, Y. Zhang, P. Yuan, Y. Tang, and T. Zhang, "Software defect prediction based on kernel PCA and weighted extreme learning machine," *Inf. Softw. Technol.*, vol. 106, pp. 182–200, Feb. 2019.

MISBAH ALI received the B.S. degree (Hons.) in computer science from the Punjab University College of Information Technology, Lahore, Pakistan, in 2015. She is currently pursuing the M.S. degree in computer science with the Virtual University of Pakistan, with a focus on software engineering. Her research interests include machine learning, data mining, and software process improvement.

TEHSEEN MAZHAR received the B.Sc. degree in computer science from Bahauddin Zakariya University, Multan, Pakistan, the M.Sc. degree in computer science from Quaid-i-Azam University, Islamabad, Pakistan, and the M.S. degree (Hons.) in computer science from the Virtual University of Pakistan, where he is currently pursuing the Ph.D. degree. He is with SED and a Lecturer with GCUF. He has more than 21 publications in well-reputed journals, such as *Electronics* (MDPI), *Health*, *Applied Science*, *Brain Sciences*, *Symmetry*, *Future Internet*, *Peer j*, and *Computers, Materials & Continua*. His research interests include machine learning, the Internet of Things, and networks.

YASIR ARIF received the B.S. degree (Hons.) in computer science from the Global Institute, Lahore, Pakistan, in 2017. His research interests include machine learning, artificial intelligence, and natural language processing.

SHAHA AL-OTAIBI (Member, IEEE) received the M.S. degree in computer science and the Ph.D. degree in artificial intelligence from King Saud University. She is currently an Associate Professor with the Department of Information Systems, College of Computer and Information Sciences, Princess Nourah bint Abdulrahman University, Saudi Arabia. Her main research interests include data science, artificial intelligence, machine learning, bio-inspired computing, cybersecurity, and information security. She is a Senior Fellow of the U.K. Higher Education Academy (SFHEA). She is a reviewer of some journals and an editorial board member of other journals.

YAZEED YASIN GHADI received the Ph.D. degree in electrical and computer engineering from Queensland University. He is currently an Assistant Professor in software engineering with Al Ain University. He was a Postdoctoral Researcher with Queensland University before joining Al Ain University. He has published more than 80 peer-reviewed journals and conference papers and holds three pending patents. His current research interests include developing novel electro-acoustic-optic neural interfaces for large-scale high-resolution electrophysiology and distributed optogenetic stimulation. He was a recipient of several awards. His dissertation on developing novel hybrid plasmonic photonic on chip-biochemical sensors received the Sigma Xi Best Ph.D. Thesis Award.

TARIQ SHAHZAD received the B.E. and M.S. degrees from COMSATS University Islamabad, Pakistan, in 2006 and 2014, respectively, and the Ph.D. degree from the University of Johannesburg, South Africa, in 2021. He is currently a Research Fellow with the University of Johannesburg, South Africa. His research work has been published in top-tier IEEE conferences and well-reputed peer-reviewed journals. His research interests include the Internet of Things, machine learning, and AI in healthcare. He has served as a technical program committee member and a paper reviewer for international conferences and journals.

MUHAMMAD AMIR KHAN received the M.Sc. degree in computer engineering from COMSATS University Islamabad, Abbottabad Campus, and the Ph.D. degree in information technology from Universiti Teknologi PETRONAS, Malaysia, with a focus on cutting-edge research. He is currently with the Department of Computer Science, Universiti Teknologi Mara, Malaysia. He is an accomplished academician and a researcher. As an Associate Professor, he continues to inspire and guide the next generation of computer scientists, leaving an indelible mark on the academic landscape. He laid the groundwork for a future dedicated to technological advancements with COMSATS University Islamabad. With an impressive record of more than 50 research papers published in ISI/Impact Factor journals and international conferences, he stands as a prominent figure shaping the discourse and advancements within the realm of computer science. His research interests include a broad spectrum, notably focusing on communication protocols for the Internet of Things (IoT), wireless sensor networks, wireless ad hoc networks, software-defined networks (SDN), and medical imaging. His contributions to these areas underscore his visionary approach to technology and his dedication to addressing contemporary challenges.

HABIB HAMAM (Senior Member, IEEE) received the B.Eng. and M.Sc. degrees in information processing from the Technical University of Munich, Germany, 1988 and 1992, respectively, and the Ph.D. degree in physics and applications in telecommunications from the University of Rennes 1 conjointly with the France Telecom Graduate School, France, in 1995, and the Postdoctoral Diploma degree "Accreditation to Supervise Research in Signal Processing and Telecommunications" from the University of Rennes 1, in 2004. From 2006 to 2016, he was the Canada Research Chair of Optics in Information and Communication Technologies, the most prestigious research position in Canada which he held for a decade. The title is awarded by the Head of the Government of Canada after a selection by an international scientific jury in the related field. He is currently a Full Professor with the Department of Electrical Engineering, University of Moncton. His research interests include optical telecommunications, wireless communications, diffraction, fiber components, RFID, information processing, the IoT, data protection, COVID-19, and deep learning. He is a Senior Member of OSA and a Registered Professional Engineer in New Brunswick. He was a recipient of several pedagogical and scientific awards. He is among others the Editor-in-Chief and the Founder of *CIT Review Journal*, an Academic Editor of *Applied Sciences*, and an Associate Editor of the IEEE CANADIAN REVIEW. He also served as a guest editor in several journals.